

Challenge1. 함수 작성 및 호출 구조 이해

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int print_banner(){
5     printf(" -- 암호 프로그램 --\n");
6 }
7 int get_shift(){
8     int shift;
9     scanf("%d", &shift);
10
11
12     while(getchar() != '\n');
13     return shift;
14 }
15
16 void encode(char *str, int shift){
17     for (int i = 0; i < strlen(str); i++){
18         if (str[i] >= 'A' && str[i] <= 'Z'){
19             str[i] = (str[i] - 'A' + shift) % 26 + 'A';
20         }
21         else if (str[i] >= 'a' && str[i] <= 'z'){
22             str[i] = (str[i] - 'a' + shift) % 26 + 'a';
23         }
24     }
25 }
```

```

int main(){
    char text[256];

    print_banner();

    printf("문자열 입력 : ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, "\n")] = 0;

    int shift = get_shift();

    encode(text, shift);

    printf("결과 : %s\n", text);

    return 0;
}

```

※왜 함수를 사용하는가?

- 똑같은 배너를 여러 번 띄우거나, 다른 문자열을 또 암호화하고 싶을 때 코드를 다시 쓸 필요 없이 함수 이름만 부르면 해결된다. 여러 번 반복하여 비슷한 동작을 실행하는 코드를 작성할 필요가 없어짐
- main 함수가 복잡해지지 않고, 코드의 흐름과 기능을 한 눈에 파악하기 편하다. 에러가 발생할 경우, 어느 기능 즉, 어느 함수에서 에러가 발생했는지를 파악하기에도 더 유용하다. 이는 추후 여러 사람과 함께 협업하여 프로그래밍해야 하는 상황에서 서로가 작성한 코드의 기능을 확인할 때 가독성에 도움을 준다.

Challenge2. gdb로 스택 프레임 관찰

- 스택 프레임이란?

함수가 실행되기 위해 잠시 빌리는 방이다. 함수 실행 도중, 새로운 함수를 만날 때 기존 함수가 아닌 새로운 함수만을 위한 작업 공간을 필요로 함. 이때, 이 작업 공간을 해당 함수의 실행이 종료되면 사라져야 하는데 이런 임시 공간을 스택 프레임이라 부른다.

```
yun@yun-VirtualBox:~$ vim caesar.c
yun@yun-VirtualBox:~$ gcc -g -O0 -o caesar caesar.c
yun@yun-VirtualBox:~$ gdb ./caesar
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./caesar...
(gdb) b main
Breakpoint 1 at 0x13ab: file caesar.c, line 23.
(gdb) b encode
Breakpoint 2 at 0x1277: file caesar.c, line 13.
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at caesar.c:23
23      int main()
```

```
(gdb) info frame
Stack level 0, frame at 0x7fffffffddd0:
  rip = 0x5555555553ab in main (caesar.c:23); saved rip = 0x7ffff7c2a1ca
  source language c.
  Arglist at 0x7fffffffddc0, args:
  Locals at 0x7fffffffddc0, Previous frame's sp is 0x7fffffffddd0
  Saved registers:
    rbp at 0x7fffffffddc0, rip at 0x7fffffffddc8
```

함수 호출 전 상태에서 info frame을 통해 확인해보면 현재 해당 함수의 main 함수 스택 프레임 상태를 알 수 있다.

rip은 현재 CPU가 실행하고 있는 코드의 위치(명령어 포인트)이다.

rbp at 0x7fffffffddc0은 현재 main 함수의 Base Pointer 위치이다.

이는 스택의 높은 주소에서 낮은 주소로 향하는 특성 때문으로, 현재 천장인 rsp는 rbp와 같거나 더 낮은 주소를 가리켜야 한다.

```
(gdb) r
Starting program: /home/yun/caesar

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at caesar.c:27
warning: Source file is more recent than executable.
27      int main(){
(gdb) c
Continuing.
-- 암호 프로그램 --
문자열 입력 : test
4

Breakpoint 2, encode (str=0x7fffffffddcb0 "test", shift=4) at caesar.c:17
17      for (int i = 0; i < strlen(str); i++){
(gdb) █
```

```
(gdb) x/16gx $rsp
0x7fffffffddc60: 0x0000000040000001      0x00007fffffffddcb0
0x7fffffffddc70: 0x00007fffffffddc90    0x00005555555527f
0x7fffffffddc80: 0x0000000040000000    0x00007fffffffdee8
0x7fffffffddc90: 0x00007fffffffddc0    0x000055555555475
0x7fffffffddca0: 0x0000000001b00000    0x0000000040000040
0x7fffffffddcb0: 0x0000000074736574    0x0000000000000004
0x7fffffffddcc0: 0x00000000208c0129    0x00007fffffffdd88
0x7fffffffddcd0: 0x00007fffffffdd00    0x00007ffff7deddb
```

call 명령어는 push rip + jmp encode가 합쳐진 기능이다.

CPU는 encode로 점프하기 직전, 다시 이전에 실행하던 곳으로 돌아오기 위해 다음으로 돌아올 함수, 즉 main함수의 주소(Return Address)를 저장한다. 이는 위 사진의 0x000555...475에서 확인할 수 있다.

```
(gdb) info frame
Stack level 0, frame at 0x7fffffffddca0:
 rip = 0x555555552b1 in encode (caesar.c:17); saved rip = 0x55555555475
 called by frame at 0x7fffffffdd0
 source language c.
 Arglist at 0x7fffffffddc90, args: str=0x7fffffffddcb0 "test", shift=4
 Locals at 0x7fffffffddc90, Previous frame's sp is 0x7fffffffddca0
 Saved registers:
  rbx at 0x7fffffffddc88, rbp at 0x7fffffffddc90, rip at 0x7fffffffddc98
```

함수 호출 후, rsp와 rbp의 값이 아까와 다른 것을 확인할 수 있다. saved rip에서는 저장된 돌아갈 주소, 즉 main함수의 주소가 저장되어 있음을 확인할 수 있다.

leave 실행 후, mov rsp, rbp와 pop rbp가 연달아 수행되며 rbp는 다시 main함수의 원래 기준으로 복구됨을 확인할 수 있다. 이때, rsp는 ret address가 들어있는 곳을 가리키게 된다.

ret 실행 직후, pop rip가 수행되며 스택에 있던 ret address를 꺼내어 원래 실행 흐름(main)으로 복귀한다. 스택 프레임이 소멸하고, rsp 역시 함수 호출 이전 상태로 정상 복구됨을 확인할 수 있다.